AI CHAT APPLICATION · REFERENCE GUIDE

Technology stack and end-to-end application flows

How the application is built and how every important request travels through it, from a click in the browser, through the guardrails and the retrieval, to the model and back. Diagrams are drawn from the code in this repository.

Contents

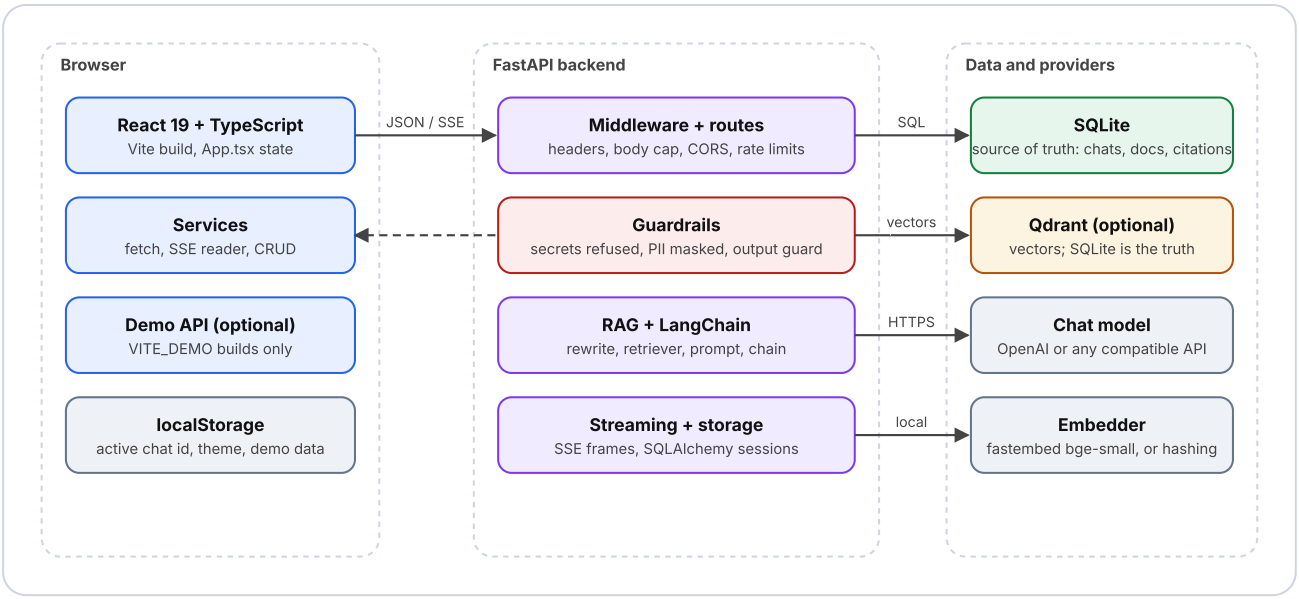
1. 1. System overview
2. 2. Technology stack
3. 3. Repository map
4. 4. Data model and vector store
5. 5. End-to-end application flows
6. 6. Security and guardrails
7. 7. Deployment
8. 8. Testing and quality gates
9. 9. Configuration reference
10. 10. API reference
11. 11. Limits and verification status

Flows

1. Startup: from 'run it' to 'chats on screen'
2. Send a message with documents (RAG, streamed, guarded)
3. Upload and index a document
4. Stop, failures and Retry
5. Delete a conversation or a document
6. Demo mode: the same app with no server
7. Delivery: from a commit to running software

1. System overview

AI Chat is a full-stack chat application. A React and TypeScript interface streams answers from a FastAPI backend. The backend calls an OpenAI-compatible model through LangChain. Conversations and uploaded documents are stored in SQLite. Answers can be grounded in the user's documents and cite them. Document vectors can optionally live in Qdrant. A separate, fully simulated demo runs on GitHub Pages with no server.



Browser

State and orchestration in `App.tsx`. Talks to the API with `fetch` and reads the SSE stream.

Backend

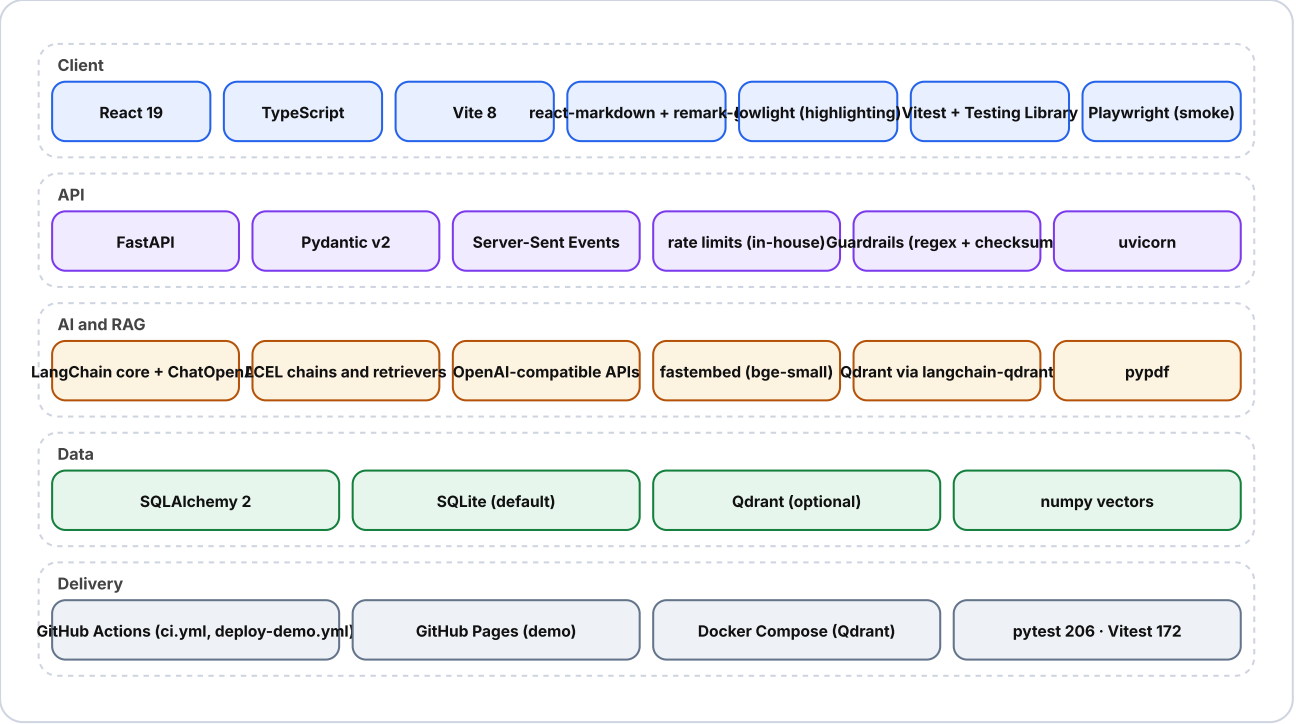
Routes validate input, run the guardrails, do the retrieval and stream the reply. Saves what was shown.

Data and providers

SQLite is the source of truth. Qdrant, the model and the embedder are replaceable parts behind small interfaces.

2. Technology stack

The stack by layer. The table below it is searchable: type a name to filter.



Layer	Technology	Version or requirement	Purpose	Where in the repo
Client	React	19 (^19.3.1)	UI components and state	frontend/src/App.tsx, components/
Client	TypeScript	7 (~7.0.2)	Typed client code	frontend/tsconfig*.json
Client	Vite	8 (^8.3.5)	Dev server and production build; VITE_* variables	frontend/vite.config.ts
Client	react-markdown + remark-gfm	10 / 4	Markdown with tables, without raw HTML	components/Markdown.tsx
Client	lowlight	3 (^3.3.0)	Syntax highlighting, rendered as React elements	lib/highlight.ts
Client	Vitest + Testing Library	Vitest 5, jsdom	Unit and component tests (172)	frontend/src/**/*.test.*
Client	Playwright	browser automation	End-to-end smoke test (e2e/smoke.cjs)	frontend/e2e/
API	FastAPI	≥ 0.115	HTTP routes, dependency	backend/main.py, routers/

			injection, SSE responses	
API	Pydantic	≥ 2.7	Request and response validation and limits	backend/schemas.py
API	uvicorn	≥ 0.30	ASGI server	run command in README
API	Guardrails (in-house)	regex + checksums	Secrets, personal data, injection warning, output masking	backend/guardrails/
AI and RAG	LangChain core, LangChain OpenAI	≥ 1.0	ChatOpenAI, prompt templates, LCEL chains, retrievers	backend/services/llm.py, retriever.py, rewrite.py
AI and RAG	OpenAI-compatible API	Chat Completions	The model: OpenAI or Groq via OPENAI_BASE_URL	.env
AI and RAG	fastembed	≥ 0.4	Local embedding model (BAAI/bge-small-en-v1.5, 384-d)	backend/services/embeddings.py
AI and RAG	Qdrant client + langchain-qdrant	≥ 1.20 / ≥ 1.1	Vector index and LangChain vector store	backend/services/qdrant_store.py
AI and RAG	pypdf	≥ 4.0	PDF text extraction, in memory	backend/services/documents.py
Data	SQLAlchemy	≥ 2.0	ORM, sessions, cascades, foreign keys	backend/database.py, models.py
Data	SQLite	standard library	Default store: chats, messages, documents, chunks, citations	backend/chat.db (created at first start)
Data	Qdrant	server 1.x (pin it)	Optional vector database	docker-compose.yml

Delivery	GitHub Actions	ci.yml, deploy-demo.yml	Checks on every push and PR; demo publish on main	.github/workflows/
Delivery	GitHub Pages	static hosting	Hosts the simulated demo	deploy-demo.yml
Delivery	Docker Compose	v2	Runs Qdrant locally or on a host	docker-compose.yml

Why these choices

- **FastAPI + Pydantic**: typed validation at the boundary; the route never runs with bad input.
- **SSE, not WebSockets**: one-way token streams over plain HTTP; the browser's built-in reader is enough.
- **SQLite first**: one file, no server, transactional, and fast enough for a single application. PostgreSQL is a configuration change later.
- **LangChain where it adds a standard interface**: the model, the chain, the retrievers and the vector store. Chunking, the prompt escaping and the citations stay in our code, because they are security- and demo-critical.
- **Qdrant as an option, not a requirement**: it adds a service. The default keeps the application runnable with no extra infrastructure.
- **Vitest and pytest**: the tests run without keys or network, so CI is deterministic.

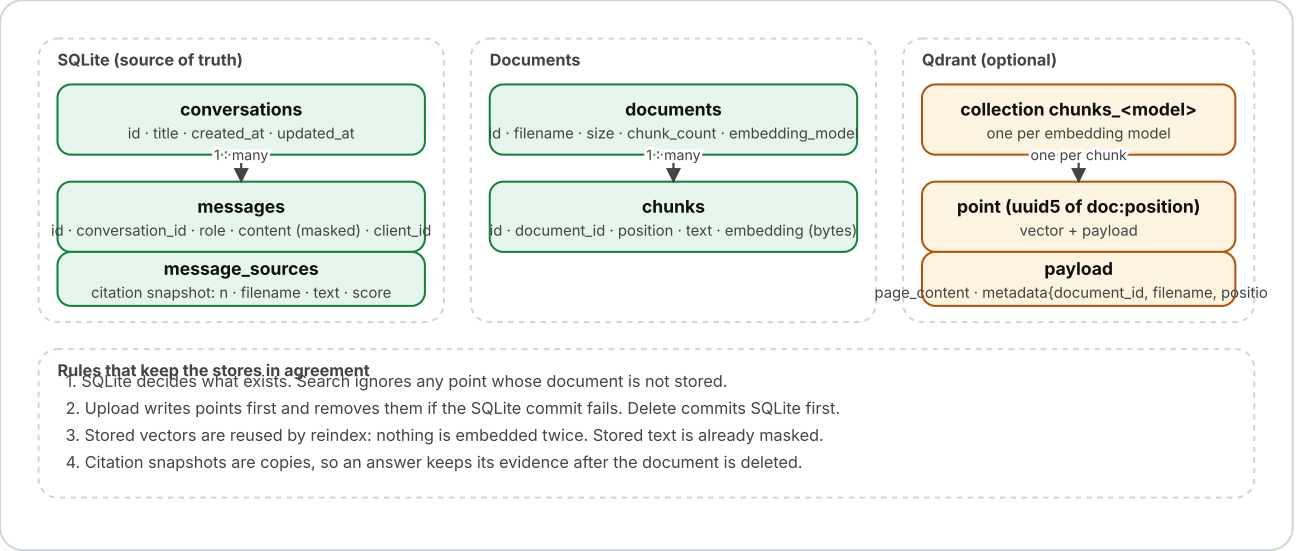
3. Repository map

Path	What it does
backend/main.py	App, middleware, CORS, /health, /info, stateless /chat endpoints
backend/routers/conversations.py	Conversations, messages (guardrails, RAG, streaming, saving)
backend/routers/documents.py	Upload, list, delete, search (index write and delete ordering)
backend/services/llm.py	LangChain chat chain, first-token priming, provider error mapping

backend/services/rag.py	Retrieval gate, relevance threshold, sources block and escaping
backend/services/rewrite.py	Standalone search query for follow-up questions
backend/services/vectorstore.py	Backend switch (sqlite | qdrant), add, delete, search, reindex, prune
backend/services/retriever.py	SqliteRetriever and QdrantRetriever (LangChain BaseRetriever)
backend/services/qdrant_store.py	Qdrant client, collections, points, embeddings adapter
backend/services/embeddings.py, chunking.py, documents.py	Embedders, chunker, text extraction
backend/guardrails/	checks.py (input, output), detectors.py, hardening.py (limits, headers, logs), policy.py
backend/streaming.py	SSE framing; saves the reply however the stream ends
backend/tools/reindex_qdrant.py	Copy stored vectors into Qdrant; prune stale entries
frontend/src/App.tsx	All state and orchestration: send, retry, regenerate, stop, open, delete, export
frontend/src/services/	http.ts (errors, demo switch), sse.ts (frame parser), chatService, documentService
frontend/src/components/	Chat UI: sidebar, messages, composer, documents panel, dialogs, markdown
frontend/src/demo/	Demo backend in the browser: store, RAG port, guardrails port, simulated assistant
frontend/e2e/smoke.cjs	Browser smoke test for the full stack and the demo build
.github/workflows/	ci.yml (checks), deploy-demo.yml (GitHub Pages)
docs/	Learning, flow, RAG, LangChain, guardrail and this guide; DEPLOY, testing; interview PDF

4. Data model and vector store

Two sets of data with one rule: SQLite decides what exists. Qdrant only answers "which chunks are close to this question".

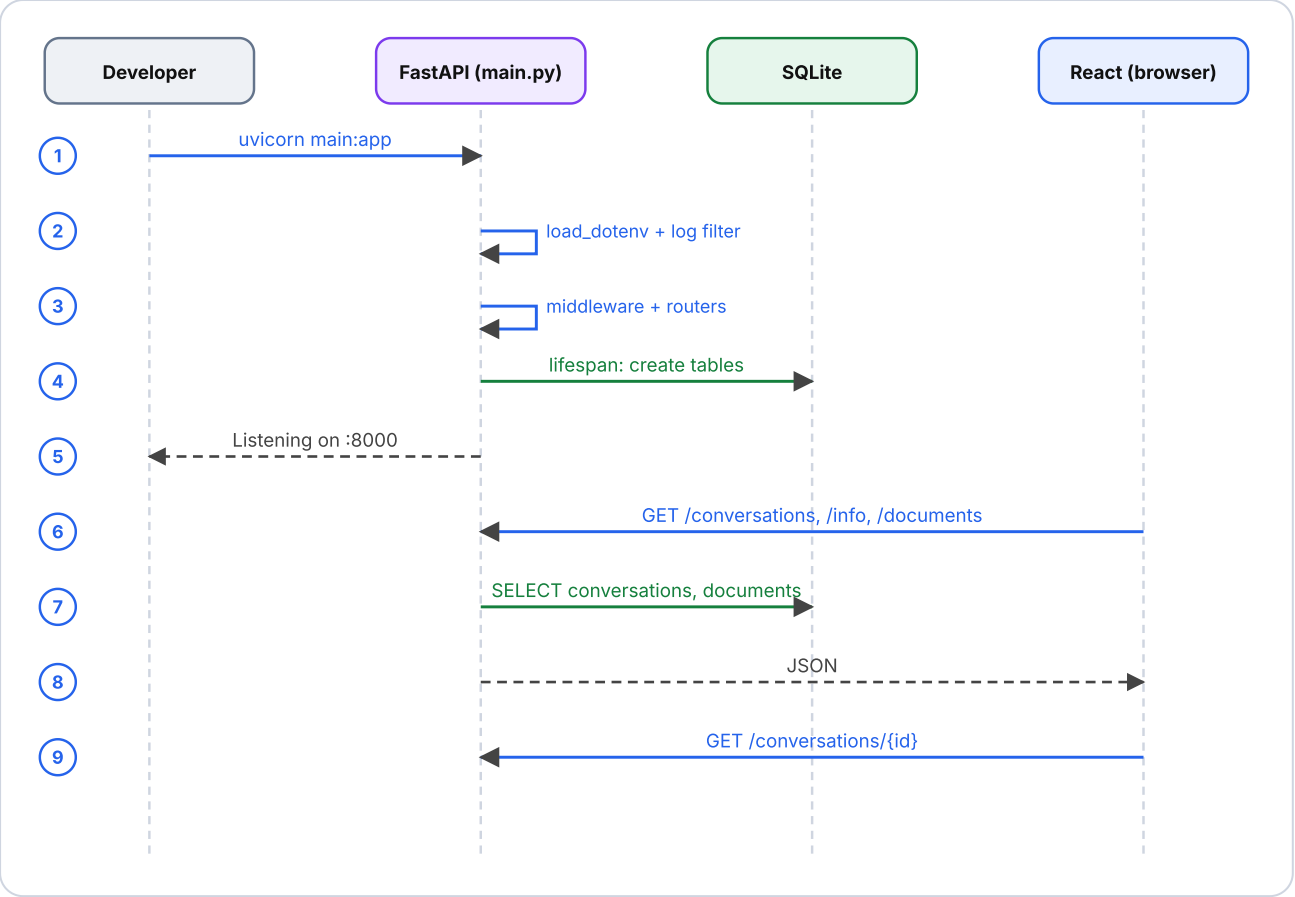


5. End-to-end application flows

Each flow is a sequence diagram. Use **Next** or **Play** to highlight one step at a time, or click a step in the list. In the PDF, every step is shown. Arrow colours: blue is a request, grey dashed is a response, green is a database write, red is a failure path.

5.1 Startup: from 'run it' to 'chats on screen'

The backend boots without an AI key or a model client; the browser restores the last chat from the API.



1 **Developer → FastAPI (main.py)**: uvicorn main:app

Uvicorn imports main.py. The module runs top to bottom, so the environment is read first.

`backend/main.py`

2 **FastAPI (main.py) → FastAPI (main.py)**: load_dotenv + log filter

backend/.env is loaded before any project import that reads the environment. The log filter that masks secrets is installed.

`main.py, guardrails/hardening.py`

3 **FastAPI (main.py) → FastAPI (main.py)**: middleware + routers

Security headers and the 2.5 MB body cap wrap everything. CORS allows CORS_ORIGINS (default localhost:5173). Routers are attached. The chat model is not created yet, so no key is needed to boot.

`main.py, services/llm.py`

4 **FastAPI (main.py) → SQLite**: lifespan: create tables

init_db() runs create_all. First start creates the tables; later starts change nothing.

`database.py`

5. **FastAPI (main.py) → Developer**: Listening on :8000
   The server accepts requests. /health works with no key configured.
   `main.py`

6. **React (browser) → FastAPI (main.py)**: GET /conversations, /info, /documents
   The page loads. The UI asks for the chat list, the deployment facts (model, provider host, guardrail policy, vector store) and the document list.
   `App.tsx, services/*`

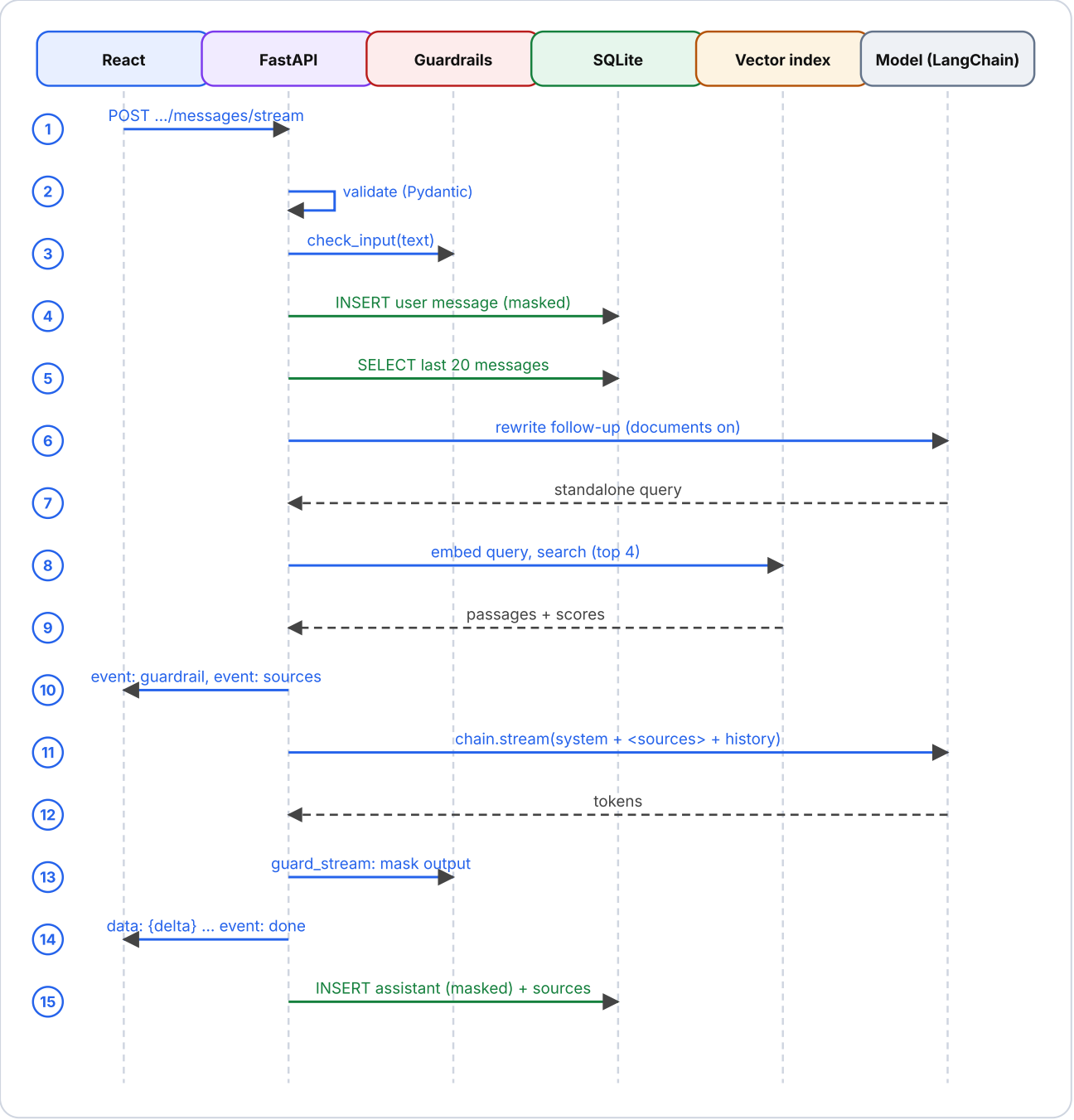
7. **FastAPI (main.py) → SQLite**: SELECT conversations, documents
   Newest 100 conversations and all documents, read from SQLite.
   `routers/*`

8. **FastAPI (main.py) → React (browser)**: JSON
   The About dialog and the model chip use /info. It never contains a key.
   `schemas.py: InfoResponse`

9. **React (browser) → FastAPI (main.py)**: GET /conversations/{id}
   The active chat id is read from localStorage, so a refresh reopens the same chat.
   `App.tsx: readStored(ACTIVE_KEY)`

5.2 Send a message with documents (RAG, streamed, guarded)

The longest path: rate limit, guardrails, storage, follow-up rewrite, retrieval, citations, streaming, masking on the way out, and saving.



1 **React → FastAPI**: POST .../messages/stream

Body is only the new text, a client_message_id and the documents switch. The message rate limit runs before the handler (429 with Retry-After when exceeded).

`routers/conversations.py`

2 **FastAPI → FastAPI**: validate (Pydantic)

1 to 4000 characters, not blank; the id is 8 to 64 characters. Failure is 422 and nothing runs.

`schemas.py`

3. **FastAPI → Guardrails**: check_input(text)

   A secret is refused: 422 with X-Guardrail: blocked, nothing stored. Personal data is masked to [REDACTED:EMAIL] and the masked text is what gets stored.

   `guardrails/checks.py`

4. **FastAPI → SQLite**: INSERT user message (masked)

   Once per client_message_id. The title is set from the first message. Committed before the model is called, so Retry is safe.

   `routers/conversations.py`

5. **FastAPI → SQLite**: SELECT last 20 messages

   The server decides the context from its own rows. A client cannot inject turns.

   `_recent_context`

6. **FastAPI → Model (LangChain)**: rewrite follow-up (documents on)

   A follow-up such as 'and how long do they nap?' becomes a standalone search query. One small call, used only for searching.

   `services/rewrite.py`

7. **Model (LangChain) → FastAPI**: standalone query

   Length-checked (500 characters). Any failure falls back to the original question.

   `rewrite.py`

8. **FastAPI → Vector index**: embed query, search (top 4)

   Embedding with fastembed (bge-small) or the offline hashing embedder. SQLite brute force, or Qdrant (which fetches 12 candidates, drops points whose document is not stored, then keeps 4). One LangChain retriever either way; the score threshold applies last.

   `services/vectorstore.py, retriever.py`

9. **Vector index → FastAPI**: passages + scores

   Best first. Each passage carries document id, filename, position and score.

   `services/retriever.py`

10. **FastAPI → React**: event: guardrail, event: sources

    Sent before any text. The bubble shows the masked text if anything was masked, and the citation chips appear.

    `streaming.py`

11. **FastAPI → Model (LangChain)**: chain.stream(system + <sources> + history)
ChatPromptTemplate | ChatOpenAI | StrOutputParser. The server's system prompt, escaped sources in the last user turn, max 800 tokens, 30 s timeout. The first piece is awaited before the response starts, so an auth or rate-limit failure is still a real HTTP status.
`services/llm.py`

12. **Model (LangChain) → FastAPI**: tokens
Text pieces stream back as the model writes them.
`_text_pieces`

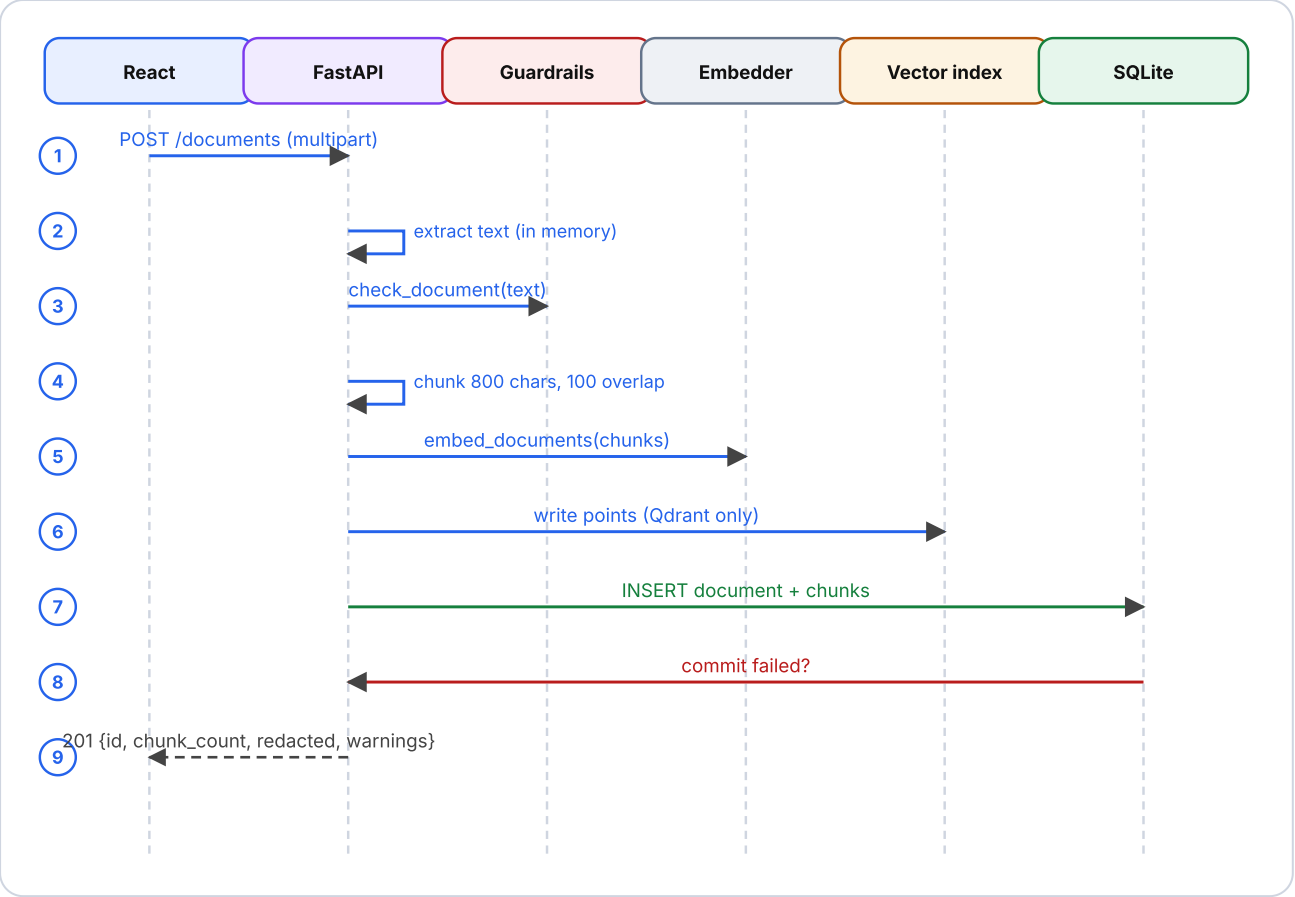
13. **FastAPI → Guardrails**: guard_stream: mask output
Secrets and personal data the model repeats are masked. The last 96 characters are held back so a split secret is still caught.
`guardrails/checks.py`

14. **FastAPI → React**: data: {delta} … event: done
Each masked piece is an SSE frame. The first frame creates the bubble. done marks completion.
`streaming.py, services/sse.ts`

15. **FastAPI → SQLite**: INSERT assistant (masked) + sources
Saved in a new session when the stream ends, is stopped or fails. The citation snapshot is copied, so it survives a document delete.
`routers/conversations.py`

5.3 Upload and index a document

Text is checked, masked, chunked and embedded before anything is stored. The index is written before the SQLite commit.



1. **React → FastAPI**: POST /documents (multipart)

   Client checks the type (.txt, .md, .pdf) and size first. Server enforces the same limits: 2 MB, 50 documents.

   `routers/documents.py`

2. **FastAPI → FastAPI**: extract text (in memory)

   UTF-8 for text and markdown; pypdf for PDF. Nothing is written to disk.

   `services/documents.py`

3. **FastAPI → Guardrails**: check_document(text)

   A secret refuses the whole file (422). Personal data is masked before chunking. Instruction-like phrases add a warning only.

   `guardrails/checks.py`

4. **FastAPI → FastAPI**: chunk 800 chars, 100 overlap

   Paragraph-aware packing; at most 400 chunks per document.

   `services/chunking.py`

5. **FastAPI → Embedder**: embed_documents(chunks)

   Unit-length vectors (384 floats for bge-small). The first call may download the model once.

   `services/embeddings.py`

6 **FastAPI → Vector index**: write points (Qdrant only)

Deterministic point ids. One collection per embedding model. SQLite backend: no-op, because the rows are written in the same transaction.

`services/vectorstore.py, qdrant_store.py`

7 **FastAPI → SQLite**: INSERT document + chunks

Chunk text and the vector bytes are stored together, then committed.

`models.py`

8 **SQLite → FastAPI**: commit failed?

If the commit fails, the new points are deleted again (best effort). Search also ignores any leftover points.

`routers/documents.py`

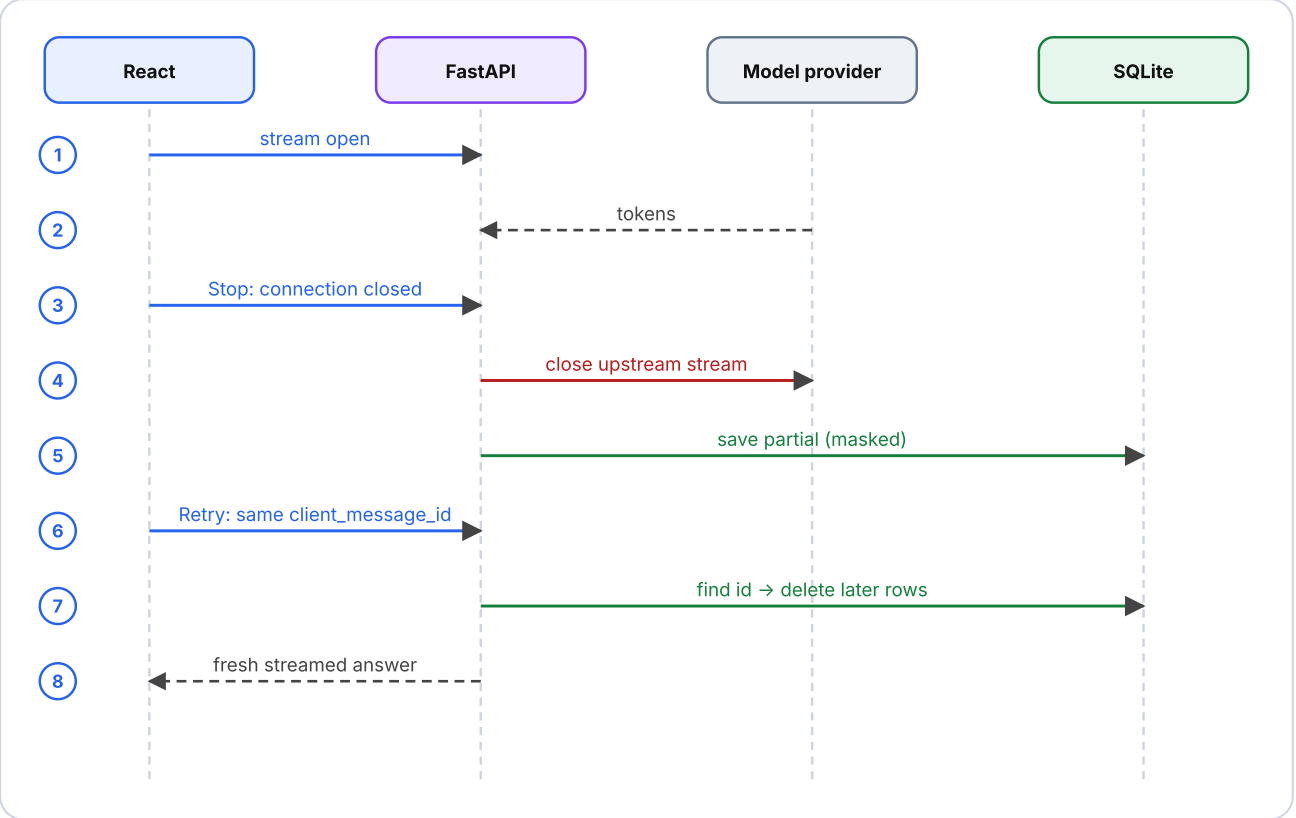
9 **FastAPI → React**: 201 {id, chunk_count, redacted, warnings}

The panel says what was masked, and any warnings. Only the upload response carries those notes.

`schemas.py`

5.4 Stop, failures and Retry

Stop keeps the partial answer and closes the provider connection. A failure before the first token is a real HTTP error. Retry re-sends the same id, so nothing is duplicated.



1. **React → FastAPI**: stream open

   Tokens are arriving and the bubble grows.

   `App.tsx: requestReply`

2. **Model provider → FastAPI**: tokens

   Pieces stream in.

   `llm.py`

3. **React → FastAPI**: Stop: connection closed

   AbortError is recognised; it is not an error. Starlette raises GeneratorExit in the response generator.

   `streaming.py`

4. **FastAPI → Model provider**: close upstream stream

   The guard's finally closes the chain's stream, which closes the HTTP connection. No more tokens are generated or billed for text nobody will read.

   `guardrails/checks.py`

5. **FastAPI → SQLite**: save partial (masked)

   The masked partial text is stored in a new session. A refresh shows exactly what was on screen.

   `streaming.py finally`

6. **React → FastAPI**: Retry: same client_message_id

   Failure before the first token: the status is real (504, 502, 429 or 500) and the UI shows an error with Retry.

   `services/llm.py: _translate`

7. **FastAPI → SQLite**: find id → delete later rows

   The stored user message is found, so no second copy. Whatever followed it (the partial answer) is deleted.

   `routers/conversations.py`

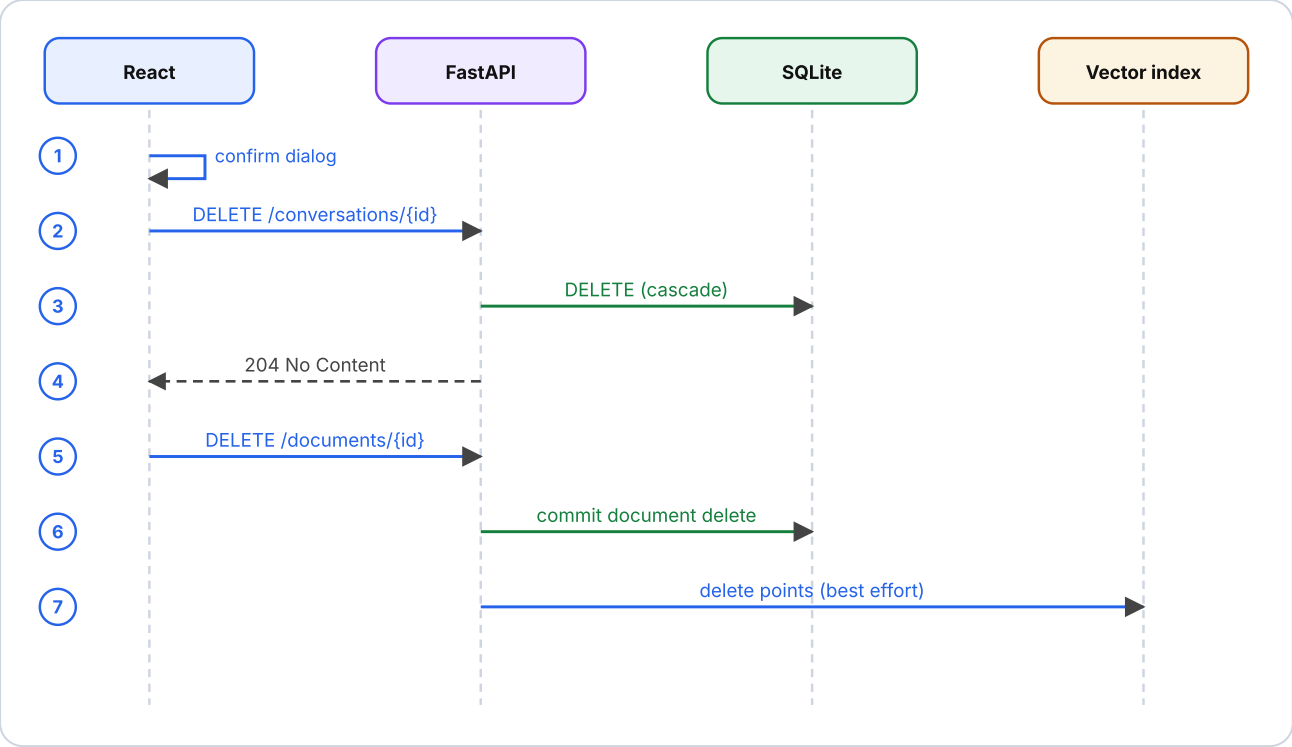
8. **FastAPI → React**: fresh streamed answer

   Exactly one user message and one answer afterwards.

   `test_conversations.py`

5.5 Delete a conversation or a document

SQLite is committed first; index entries are removed afterwards and are never returned even if that step fails.



1. **React → React**: confirm dialog

   Accessible modal (focus trap, Escape). Nothing is sent until the user confirms.

   `components/ConfirmDialog.tsx`

2. **React → FastAPI**: DELETE /conversations/{id}

   Also used by the cleanup of a chat created by a refused message.

   `routers/conversations.py`

3. **FastAPI → SQLite**: DELETE (cascade)

   Messages and their citation snapshots go with the chat, by ORM cascade and by foreign key.

   `models.py`

4. **FastAPI → React**: 204 No Content

   The list refreshes. Other chats are untouched.

   `App.tsx`

5. **React → FastAPI**: DELETE /documents/{id}

   Deleting a document removes its chunks in SQLite.

   `routers/documents.py`

6. **FastAPI → SQLite**: commit document delete

   The source of truth changes first.

   `routers/documents.py`

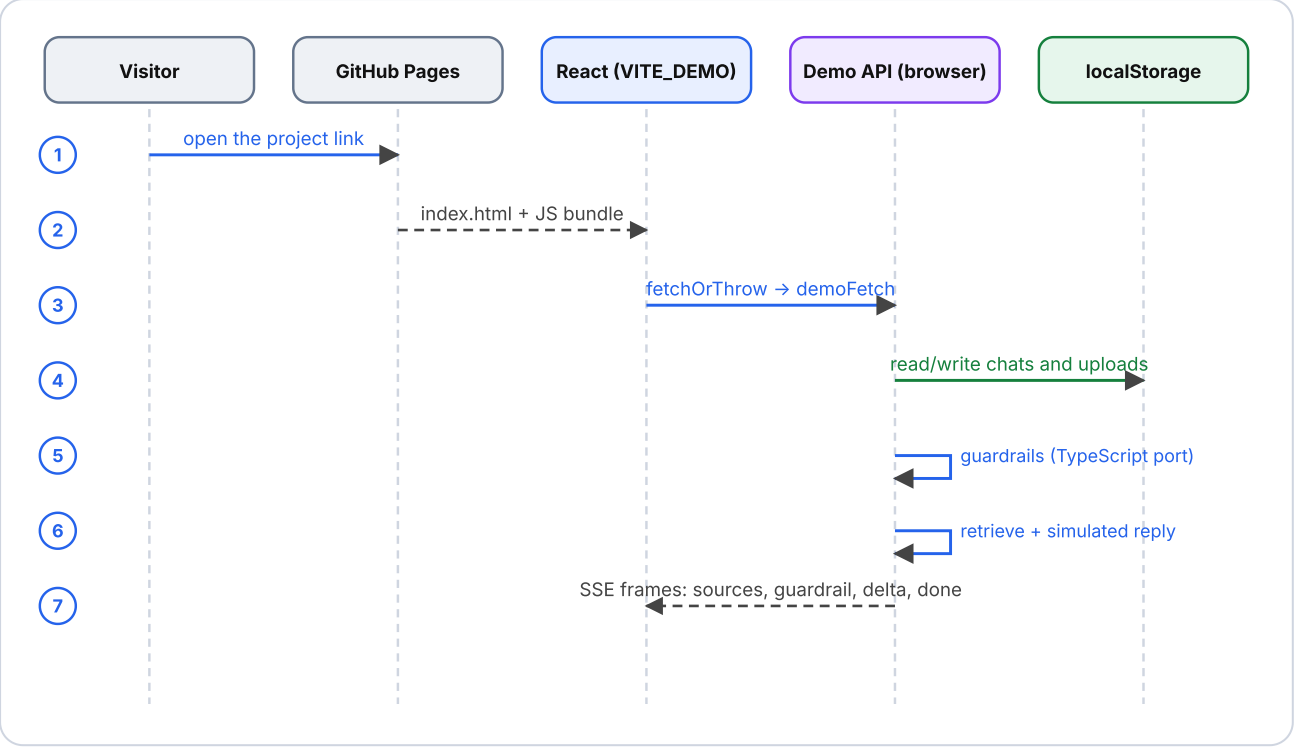
7 **FastAPI → Vector index**: delete points (best effort)

If this fails the response is still 204. Search hides the leftovers; reindex_qdrant.py --prune removes them.

`services/vectorstore.py`

5.6 Demo mode: the same app with no server

The public GitHub Pages build answers its own API calls in the browser, with simulated replies and no key, no shared database and no backend.



1 **Visitor → GitHub Pages**: open the project link

A static build of the frontend is served. Nothing dynamic runs on GitHub's side.

`deploy-demo.yml`

2 **GitHub Pages → React (VITE_DEMO)**: index.html + JS bundle

Built with VITE_DEMO=true and VITE_BASE set to the repository path.

`vite.config.ts`

3 **React (VITE_DEMO) → Demo API (browser)**: fetchOrThrow → demoFetch

The same services and SSE parser run. Only the transport is swapped (dynamic import, so the demo code is not in the main bundle).

`services/http.ts`

4. **Demo API (browser) → localStorage**: read/write chats and uploads

   All state is in this browser. Nothing is sent to a server.

   `demo/store.ts`

5. **Demo API (browser) → Demo API (browser)**: guardrails (TypeScript port)

   Refuses secrets and masks personal data, tested against the Python detectors on a shared fixture.

   `demo/guardrails.ts`

6. **Demo API (browser) → Demo API (browser)**: retrieve + simulated reply

   Client-side RAG: the chunking and hashing embedder are ports, tested for parity. The answer is composed from the passages and streamed word by word.

   `demo/rag.ts, assistant.ts`

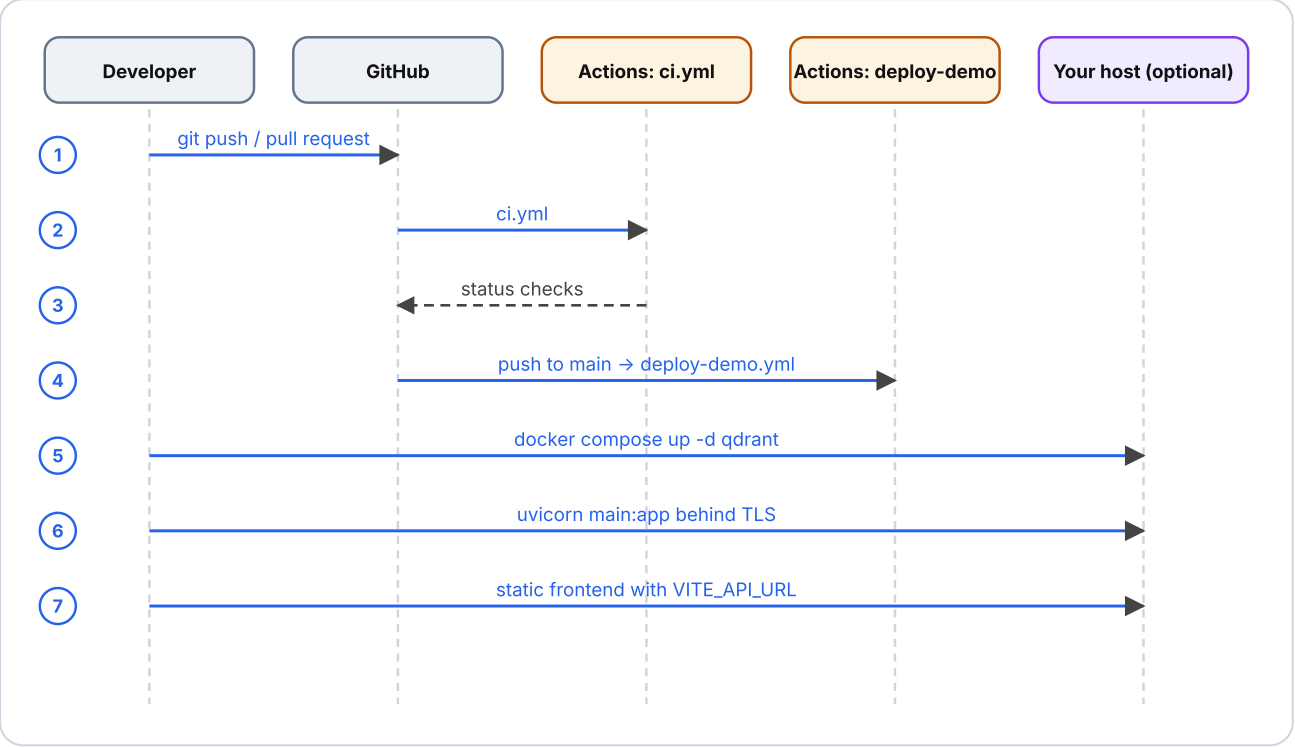
7. **Demo API (browser) → React (VITE_DEMO)**: SSE frames: sources, guardrail, delta, done

   Streamed exactly like the real server, so the UI code path is the same.

   `demo/api.ts`

5.7 Delivery: from a commit to running software

Pushes and pull requests run the checks. A push to main publishes the demo. The full stack is deployed by you, with the key and vector store configured on the host.



1. **Developer → GitHub**: git push / pull request
   Any branch and any pull request.
   `GitHub`

2. **GitHub → Actions: ci.yml**: ci.yml
   Backend: pip install, pytest (206). Frontend: npm ci, lint, tsc, vitest (172), build.
   `ci.yml`

3. **Actions: ci.yml → GitHub**: status checks
   Pass or fail on the commit. The tests need no keys, network or model download.
   `ci.yml`

4. **GitHub → Actions: deploy-demo**: push to main → deploy-demo.yml
   Build with VITE_DEMO=true, then upload the Pages artifact and deploy it.
   `deploy-demo.yml`

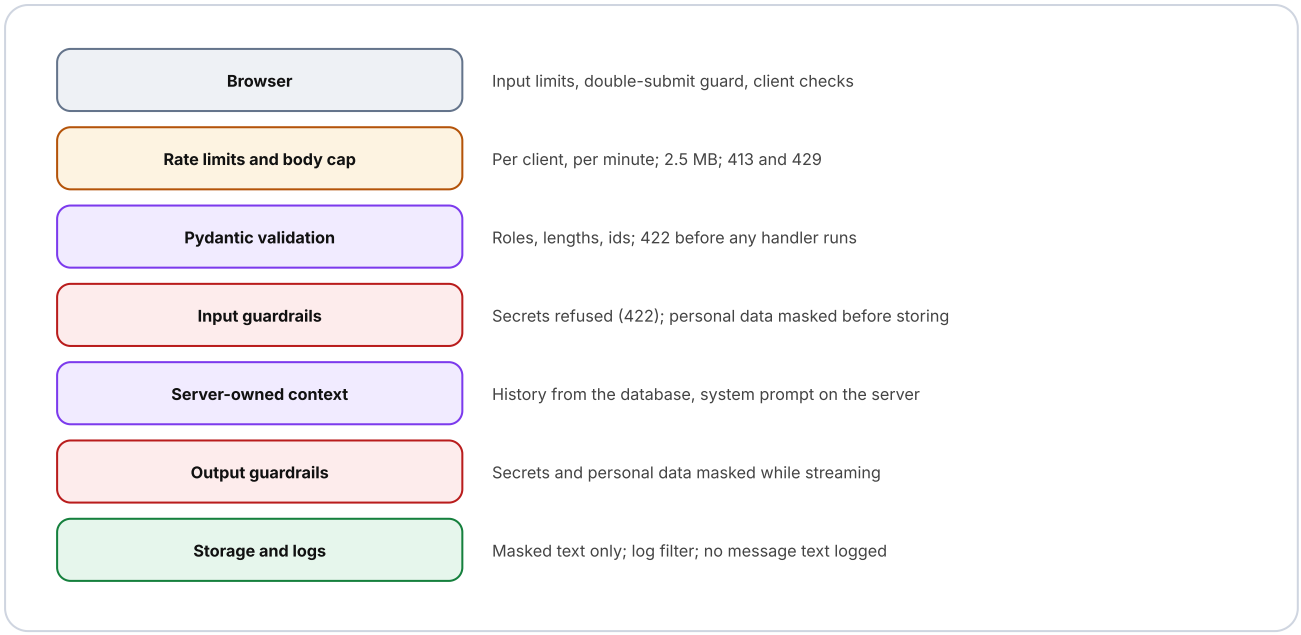
5. **Developer → Your host (optional)**: docker compose up -d qdrant
   The optional Qdrant server, bound to 127.0.0.1 and protected by QDRANT_API_KEY.
   `docker-compose.yml`

6. **Developer → Your host (optional)**: uvicorn main:app behind TLS
   Set CORS_ORIGINS to the frontend origin, VECTOR_STORE, provider key, and TRUST_PROXY only behind your proxy.
   `docs/DEPLOY.md`

7. **Developer → Your host (optional)**: static frontend with VITE_API_URL
   The frontend build points at the API. Public variables only, never secrets.
   `frontend/.env`

6. Security and guardrails

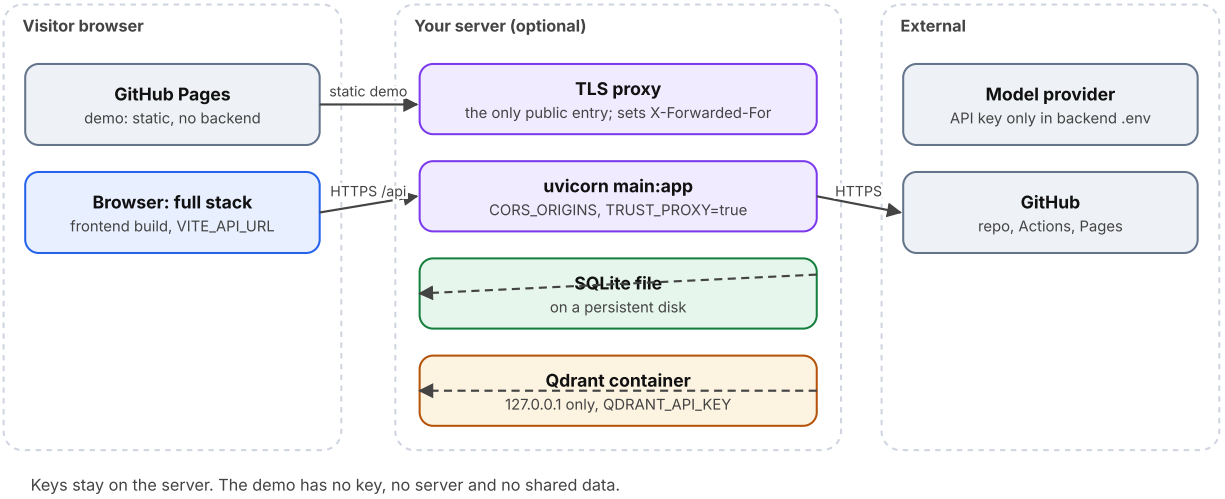
Defence in layers. Each layer stops one class of problem, and none is treated as complete on its own.



Concern	Control	Where
Secrets typed or pasted	Refused with 422; the text returns to the box; nothing stored or sent	guardrails/checks.py
Personal data typed or uploaded	Masked before storing, chunking and embedding	guardrails/checks.py, routers/documents.py
Secrets or personal data in a reply	Masked while streaming; the saved reply is the masked one	guardrails/checks.py: guard_stream
Prompt injection through documents	Escaped <sources> block in the user turn; system prompt says sources are untrusted; a warning on upload	services/rag.py
Floods and oversized bodies	Rate limits (429) and a 2.5 MB cap (413)	guardrails/hardening.py
Leaking provider errors	Fixed, friendly messages; upstream text never returned	services/llm.py: _translate
Browser attacks on replies	Markdown without raw HTML, images or javascript: links; security headers on the API	Markdown.tsx, hardening.py
Vector store exposure	Qdrant bound to localhost, API key required by compose; no per-user access control	docker-compose.yml
No authentication	Not built. Required before public deployment	docs/DEPLOY.md

7. Deployment

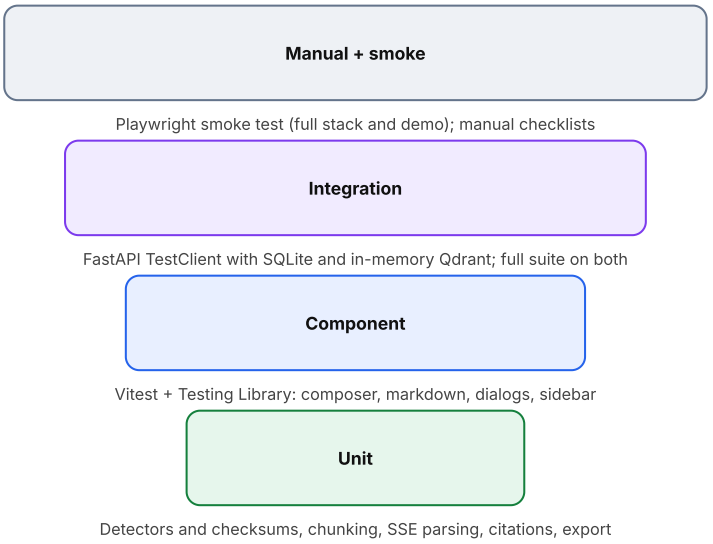
Two targets. The demo is safe to publish because it has nothing to protect. The full stack holds a key and user data and is run by you, behind TLS, with the settings below.



Full-stack checklist

1. Set `OPENAI_API_KEY` and `OPENAI_MODEL` in the server's environment, never in the frontend build.
2. Set `CORS_ORIGINS` to the exact frontend origin.
3. If Qdrant: set `QDRANT_API_KEY`, run `docker compose up -d qdrant`, set `VECTOR_STORE=qdrant` and `QDRANT_URL`, then run `tools/reindex_qdrant.py --prune`.
4. Keep the database and the Qdrant volume on persistent disks; back up both.
5. Set `TRUST_PROXY=true` only if your own proxy overwrites `X-Forwarded-For`.
6. Run the smoke test against the deployed site before announcing it.

8. Testing and quality gates



Gate	Command	What it proves	Count
Backend tests (SQLite)	pytest -q	Routes, guardrails, streaming, RAG, storage	206
Backend tests (Qdrant)	TEST_VECTOR_STORE=qdrant pytest -q	The same suite against an in-memory Qdrant	206
Frontend lint and types	npm run lint; npx tsc -b	Static checks	—
Frontend tests	npm test	Components, parsers, the demo port and its parity with the Python guardrails	172
Production build	npm run build	Bundles compile; demo variant with VITE_DEMO	—
Browser smoke	node e2e/smoke.cjs <url> [demo]	Send, regenerate, refresh, export, shortcuts, citations	—

CI pipeline

`ci.yml` runs the backend and frontend gates on every push and pull request. `deploy-demo.yml` builds the demo on pushes to `main` and publishes it to GitHub Pages.

9. Configuration reference

Variable	Where	Meaning
OPENAI_API_KEY	backend/.env	Provider key. Server only. Never logged.
OPENAI_MODEL	backend/.env	Model name for the provider.
OPENAI_BASE_URL	backend/.env	Empty for OpenAI; https://api.groq.com/openai/v1 for Groq.
DATABASE_URL	backend/.env	Default sqlite:///backend/chat.db. A PostgreSQL URL works with its driver installed.
CORS_ORIGINS	backend/.env	Comma-separated frontend origins. Default localhost:5173.
EMBEDDING_BACKEND / EMBEDDING_MODEL	backend/.env	fastembed (default) or hashing (offline, tests).
VECTOR_STORE	backend/.env	sqlite (default) or qdrant.
QDRANT_URL / QDRANT_API_KEY / QDRANT_PATH	backend/.env	Server URL and key, or an embedded on-disk store.
RAG_MIN_SCORE	backend/.env	Relevance threshold. 0.5 for bge-small (a starting point, to be tuned).
RAG_REWRITE_QUERIES	backend/.env	true by default; false disables the follow-up rewrite call.
GUARDRAILS, GUARDRAIL_SECRETS, GUARDRAIL_PII	backend/.env	on; block; redact (the defaults).
RATE_LIMIT_*_PER_MINUTE	backend/.env	Per client: messages 30, uploads 10, search 30, chat 30, new chats 60. 0 turns off.
MAX_REQUEST_BYTES, TRUST_PROXY	backend/.env	2.5 MB cap. TRUST_PROXY only behind your own reverse proxy.
VITE_API_URL	frontend/.env	Public. Where the built frontend finds the API.
VITE_DEMO, VITE_BASE	build environment	Demo build and the sub-path for GitHub Pages.

10. API reference

Method	Path	Database effect	Notes
GET	/health	none	200 {status: ok}
GET	/info	none	Model, provider host, guardrail policy, vector store, limits
POST	/chat, /chat/stream	none	Stateless (client sends history); guarded; rate limited
GET	/conversations	SELECT	Newest 100
POST	/conversations	INSERT	Rate limited (60 a minute)
GET	/conversations/{id}	SELECT	Chat and messages (masked text)
DELETE	/conversations/{id}	DELETE	Cascade to messages and citations
POST	/conversations/{id}/messages/stream	INSERT, SELECT, INSERT	SSE: guardrail, sources, delta, done, error
GET	/documents	SELECT	Documents, newest first
POST	/documents	INSERT	Multipart upload; 201 with redacted and warnings
DELETE	/documents/{id}	DELETE	SQLite first, then index entries
POST	/documents/search	SELECT	Top k with scores (no threshold)

Interactive documentation is served by FastAPI at `/docs` when the backend runs (not served in production, and not covered by the security header rules).

11. Limits and verification status

What this guide and the test suite establish, and what they do not. Read this before relying on a claim above.

- Detection is patterns and checksums. Names, addresses and unfamiliar secret formats are not caught.
- Authentication is not built. Anyone who can reach the API can read every conversation. Required before any public deployment.
- Rate limits are per client address and per process. Several workers each keep their own count.
- Qdrant server mode has not been run in this project's environment (no Docker daemon). The embedded and in-memory modes are tested.
- The default embedder (bge-small) has not been measured here: model download is blocked in the build environment. The threshold of 0.5 is a starting point.
- Follow-up rewriting is tested with a scripted model. Its quality with a real model is unmeasured.
- The GitHub Actions workflows have not run on GitHub from this environment. They are validated as YAML and locally by their commands.
- The demo build is simulated: it has no real model and no output guard on a real reply.

AI Chat application · generated from the repository's code and manifests · see docs/AI-Chat-Flows-Guide.html for the code-level walkthrough